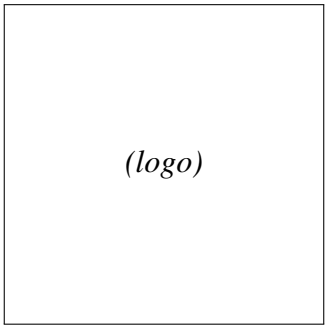
ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA HỆ THỐNG THÔNG TIN



BÁO CÁO ĐỒ ÁN

MÔN KHO DỮ LIỆU VÀ OLAP — IS217.R13

XÂY DỰNG KHO DỮ LIỆU VÀ PHÂN TÍCH HIỆU QUẢ KINH DOANH TRÊN NỀN TẢNG THƯƠNG MẠI ĐIỆN TỬ

GIẢNG VIÊN HƯỚNG DẪN

NGUYỄN THỊ KIM PHỤNG

SINH VIÊN THỰC HIỆN

NGUYỄN THÚY VY — 24522071

PHẠM QUANG HỮU PHƯỚC — 24521411

TP. HỒ CHÍ MINH, 9/2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

LỜI CẢM ƠN

Lời đầu tiên, nhóm thực hiện đề tài xin gửi tới Cô Nguyễn Thị Kim Phụng, Giảng viên môn Kho dữ liệu và OLAP lời cảm ơn chân thành và sâu sắc nhất.

Chúng em xin cảm ơn Cô vì những tri thức mà Cô truyền đạt đến chúng em một cách hăng say và đầy nhiệt huyết. Chúng em cảm ơn Cô vì những lời khuyên, hướng dẫn và góp ý đã giúp chúng em có được định hướng đúng đắn cho đề tài của nhóm.

Trong quá trình thực hiện đồ án, nhóm chúng em đã vận dụng những kiến thức được Cô cung cấp tại lớp, đồng thời chủ động học hỏi và tiếp thu từ nhiều nguồn tài liệu khác nhau để có thể thực hiện đồ án này một cách tốt nhất. Tuy nhiên, do chưa có kinh nghiệm thực tế đồ án vẫn không tránh khỏi những điểm thiếu sót, bất cập. Vì vậy, chúng em mong rằng Cô sẽ đưa ra nhận xét, đánh giá và góp ý để chúng em có thể củng cố kiến thức cũng như rút kinh nghiệm để nhóm có thể thực hiện các đề tài trong tương lai tốt hơn và hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện.

Nguyễn Thúy Vy,

Phạm Quang Hữu Phước

MỤC LỤC

1 Giới Thiệu **10**

1.1 Lý do chọn đề tài . . . 10

1.2 Giới thiệu về dataset . . . 10

1.2.1 Tổng quan dataset . . . 10

1.2.2 Mô tả quy mô và thời gian thu thập . . . 10

1.3 Quy trình tiền xử lý dữ liệu . . . 11

1.3.1 Khám phá và phát hiện vấn đề dữ liệu . . . 11

1.3.1.1 Kiểm tra số lượng dòng và khóa chính . . . 12

1.3.1.2 Kiểm tra dữ liệu nhiều dòng trên cùng một mã đơn hàng ở bảng thanh toán và đánh giá . . . 12

1.3.2 Xử lý giá trị thiếu . . . 13

1.3.2.1 Rà soát tổng quan giá trị thiếu trên tất cả bảng thô . . . 13

1.3.2.2 Thay thế giá trị thiếu cột nguồn gốc trong bảng khách hàng tiềm năng . . . 14

1.3.2.3 Giữ nguyên giá trị thiếu các cột biểu mẫu marketing trong bảng chốt hợp đồng . . . 15

1.3.2.4 Thay thế giá trị thiếu tiêu đề và nội dung nhận xét trong bảng đánh giá . . . 15

1.3.2.5 Giữ nguyên giá trị thiếu các cột ngày giao hàng và chuyển đổi kiểu dữ liệu . . . 16

1.3.2.6 Xử lý giá trị thiếu theo từng nhóm cột trong bảng sản phẩm . 17

1.3.2.7 Các bảng không có giá trị thiếu . . . 17

1.3.3 Kiểm tra và xử lý dữ liệu trùng lặp . . . 18

1.3.4 Kiểm tra tính toàn vẹn tham chiếu trên dữ liệu thô . . . 19

1.3.4.1 Kiểm tra sự đầy đủ của danh mục sản phẩm trong bảng dịch danh mục . . . 19

1.3.4.2 Kiểm tra sự tồn tại của mã người bán từ bảng chốt hợp đồng trong bảng người bán . . . 19

1.3.4.3 Kiểm tra tính khép kín của mã khách hàng tiềm năng . . . 20

1.3.5 Chuẩn hóa mã bưu chính (Zip Code Prefix) . . . 20

1.4 Thiết kế kho dữ liệu 20
1.4.1 Lựa chọn mô hình lược đồ 20
1.4.2 Lược đồ kho dữ liệu 21
1.4.3 Xây dựng và mô tả chi tiết các bảng Chiều (Dimension Tables) 22
1.4.3.1 Bảng chiều địa điểm (DIM_LOCATION) 22
1.4.3.2 Bảng chiều danh mục sản phẩm (DIM_PRODUCT_CATEGORY) 23
1.4.3.3 Bảng chiều sản phẩm (DIM_PRODUCT) 23
1.4.3.4 Bảng chiều khách hàng (DIM_CUSTOMER) 24
1.4.3.5 Bảng chiều người bán (DIM_SELLER) 26
1.4.3.6 Bảng chiều đánh giá (DIM_REVIEW) 27
1.4.4 Xây dựng và mô tả chi tiết các bảng Sự kiện (Fact Tables) 28
1.4.4.1 Bảng sự kiện chi tiết đơn hàng (FACT_ORDER_ITEMS) . . 28
1.4.4.2 Bảng sự kiện thanh toán (FACT_ORDER_PAYMENT) . . . 29
1.4.4.3 Bảng sự kiện khách hàng tiềm năng (FACT_SELLER_LEAD) 30
1.4.5 Kiểm tra tính toàn vẹn tham chiếu 31

2 Quá trình xây dựng kho dữ liệu (SSIS) 33

2.1 Chuẩn bị các công cụ 33
2.2 Chuẩn bị cơ sở dữ liệu 33
2.3 Tạo mới project SSIS 33
2.4 Tạo bảng DIM và FACT 33
2.4.1 Load dữ liệu gốc từ files .csv vào Data Flow 33
2.4.2 Bảng DIM_Time 33
2.4.3 Bảng DIM_Language 33
2.4.4 Bảng DIM_Artist 33
2.4.5 Bảng DIM_Album 33
2.4.6 Bảng DIM_AudioFeature 33
2.4.7 Bảng FACT_Track 33
2.4.8 Tạo khóa ngoại từ bảng FACT đến các bảng DIM 33

2.4.9 Chạy dự án SSIS 33
2.5 Kiểm tra dữ liệu các bảng 33
2.6 Lược đồ sau khi hoàn thành 33

DANH MỤC HÌNH ẢNH

1.1 Lược đồ quan hệ của bộ dữ liệu Olist gốc gồm 11 bảng 11

1.2 Kiểm tra số lượng dòng và tính hợp lệ của khóa chính trên các bảng thô 12

1.3 Kiểm tra phân bố nhiều dòng dữ liệu trên cùng một mã đơn hàng 12

1.4 Tổng quan phân bố giá trị thiếu trên tất cả bảng dữ liệu thô 14

1.5 Thay thế giá trị thiếu cột nguồn gốc tiếp thị bằng placeholder 'unknown' 14

1.6 Thống kê và bảo toàn giá trị thiếu các cột biểu mẫu marketing trong bảng chốt hợp đồng 15

1.7 Thay thế giá trị thiếu tiêu đề và nội dung nhận xét bằng chuỗi rỗng 16

1.8 Bảo toàn giá trị thiếu các mốc thời gian giao hàng sau khi chuyển đổi kiểu dữ liệu 16

1.9 Hiện thực hóa xử lý giá trị thiếu trên các nhóm cột trong bảng sản phẩm 17

1.10 Rà soát bản ghi trùng lặp theo khóa chính và toàn dòng trên tất cả bảng thô . . 18

1.11 Khử trùng lặp khóa chính review_id trong bảng đánh giá 18

1.12 Kiểm tra sự đầy đủ của danh mục sản phẩm trong bảng dịch danh mục 19

1.13 Kiểm tra đối chiếu mã người bán giữa bảng chốt hợp đồng và bảng người bán . 19

1.14 Xác nhận tính khép kín của định danh khách hàng tiềm năng trong phễu tiếp thị 20

1.15 Chuẩn hóa mã bưu chính về chuỗi ký tự cố định 5 chữ số 20

1.16 Lược đồ bông tuyết của kho dữ liệu Olist sau tiền xử lý 22

1.17 Hiện thực hóa xây dựng bảng chiều địa điểm DIM_LOCATION 22

1.18 Xây dựng bảng chiều danh mục sản phẩm song ngữ 23

1.19 Cấu trúc bảng chiều sản phẩm sau khi làm sạch 24

1.20 Quy trình ánh xạ khóa địa điểm cho bảng chiều khách hàng (Bước 1) 25

1.21 Thực hiện fallback lookup và bổ sung địa danh cho khách hàng (Bước 2) 25

1.22 Xác nhận toàn vẹn khóa ngoại location_id trên bảng khách hàng (Bước 3) . . . 26

1.23 Ánh xạ khóa địa điểm cho bảng chiều người bán (Phần 1) 26

1.24 Ánh xạ khóa địa điểm cho bảng chiều người bán (Phần 2) 27

1.25 Khẳng định tính hoàn thiện không khuyết thiếu của khóa ngoại ở bảng chiều người bán 27

1.26 Cấu trúc hoàn thiện của bảng chiều đánh giá DIM_REVIEW 28

1.27 Xây dựng bảng sự kiện chi tiết đơn hàng FACT_ORDER_ITEMS 29
1.28 Xây dựng bảng sự kiện thanh toán FACT_ORDER_PAYMENT 29
1.29 Xây dựng bảng sự kiện phễu tiếp thị FACT_SELLER_LEAD 30
1.30 Kiểm tra tính toàn vẹn tham chiếu giữa các bảng sự kiện và bảng chiều 31

DANH MỤC BẢNG

1.1 Chiến lược xử lý giá trị thiếu (NULL) theo nhóm đặc trưng 13
1.2 Quy tắc xử lý giá trị thiếu theo từng nhóm cột trong bảng sản phẩm 17
1.3 Các thuộc tính của bảng DIM_LOCATION 23
1.4 Các thuộc tính của bảng DIM_PRODUCT_CATEGORY 23
1.5 Các thuộc tính của bảng DIM_PRODUCT 24
1.6 Các thuộc tính của bảng DIM_CUSTOMER 26
1.7 Các thuộc tính của bảng DIM_SELLER 27
1.8 Các thuộc tính của bảng DIM_REVIEW 28
1.9 Các thuộc tính của bảng FACT_ORDER_ITEMS 29
1.10 Các thuộc tính của bảng FACT_ORDER_PAYMENT 30
1.11 Các thuộc tính của bảng FACT_SELLER_LEAD 30

1 Giới Thiệu

1.1 Lý do chọn đề tài

Trong những năm gần đây, thương mại điện tử ngày càng phát triển và trở thành một phần quan trọng trong hoạt động kinh doanh. Cùng với sự phát triển đó, các nền tảng thương mại điện tử tạo ra một lượng lớn dữ liệu liên quan đến khách hàng, đơn hàng, sản phẩm, người bán, thanh toán và vận chuyển. Nguồn dữ liệu này không chỉ phản ánh tình hình kinh doanh mà còn chứa nhiều thông tin có giá trị cho việc đánh giá và đưa ra các quyết định phù hợp.

Tuy nhiên, khi dữ liệu được lưu trữ ở nhiều nguồn và dưới nhiều dạng khác nhau, việc tổng hợp và khai thác trực tiếp trên hệ thống giao dịch gặp nhiều khó khăn. Do đó, việc xây dựng một kho dữ liệu tập trung giúp tổ chức, lưu trữ và quản lý dữ liệu một cách có hệ thống, đồng thời tạo nền tảng thuận lợi cho quá trình phân tích và khai thác thông tin.

Xuất phát từ những lý do trên, nhóm lựa chọn đề tài **"Xây dựng kho dữ liệu và phân tích hiệu quả kinh doanh trên nền tảng thương mại điện tử"** dựa trên bộ dữ liệu Olist. Thông qua đề tài, nhóm hướng đến việc xây dựng kho dữ liệu phục vụ phân tích các khía cạnh như doanh thu, đơn hàng, sản phẩm, khách hàng và người bán theo nhiều chiều dữ liệu khác nhau. Qua đó, đề tài giúp vận dụng các kiến thức về **Kho dữ liệu và OLAP** vào một bài toán thực tế, đồng thời tạo ra cơ sở để tổng hợp và khai thác các thông tin phục vụ đánh giá hiệu quả hoạt động kinh doanh trên nền tảng thương mại điện tử.

1.2 Giới thiệu về dataset

1.2.1 Tổng quan dataset

Tập dữ liệu thương mại điện tử Olist (Brazil) dạng cơ sở dữ liệu quan hệ SQLite, lưu trữ toàn bộ chu trình kinh doanh trực tuyến thực tế: từ đơn hàng, thông tin khách hàng, người bán, danh mục sản phẩm, phương thức thanh toán, vận chuyển cho đến đánh giá (reviews).

1.2.2 Mô tả quy mô và thời gian thu thập

Nguồn dữ liệu: Bộ dữ liệu được công bố công khai trên Kaggle: E-commerce dataset by Olist as an SQLite database.

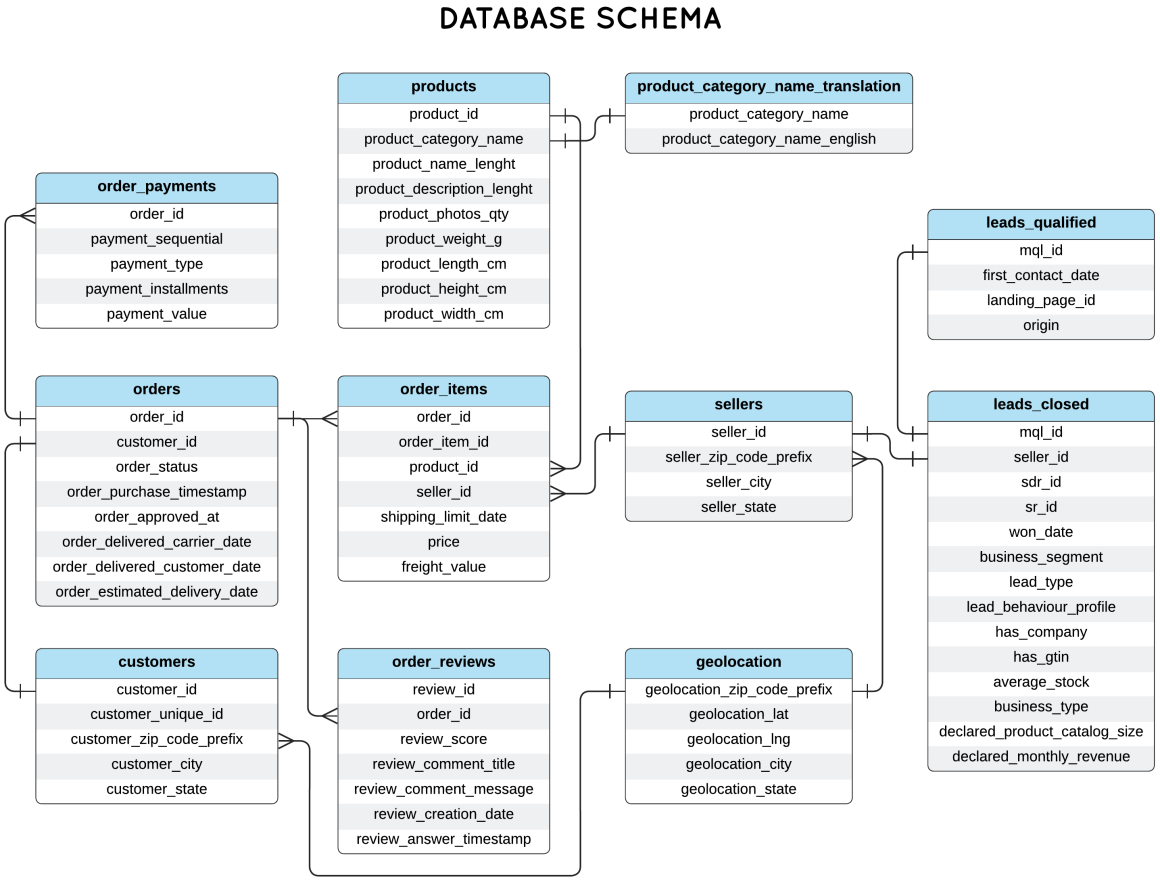
Thời gian thu thập: Từ tháng 09/2016 đến tháng 10/2018 (đơn hàng sớm nhất ngày 04/09/2016, đơn hàng muộn nhất ngày 17/10/2018).

Cấu trúc: Gồm **11 bảng quan hệ** (orders, customers, order_items, order_payments, order_reviews, products, product_category_name_translation, sellers, geolocation, leads_qualified, leads_closed) liên kết với nhau qua các khóa chính và khóa ngoại.

Số dòng: Tổng cộng **1.559.764 dòng** trên toàn bộ 11 bảng, trong đó có **99.441 đơn hàng** và **112.650 dòng chi tiết đơn hàng**; riêng bảng tọa độ geolocation chiếm 1.000.163 dòng.

Số cột: Tổng cộng **70 cột thuộc tính** phân bổ trên 11 bảng (mỗi bảng từ 2 đến 14 cột).

Cấu trúc ban đầu của bộ dữ liệu cùng quan hệ giữa các bảng được thể hiện trong Hình 1.1.



Hình 1.1: Lược đồ quan hệ của bộ dữ liệu Olist gốc gồm 11 bảng

1.3 Quy trình tiền xử lý dữ liệu

Toàn bộ quy trình tiền xử lý dữ liệu được thực hiện bằng Python (sử dụng hai thư viện chính là pandas và numpy), xử lý dữ liệu thô từ các tệp CSV trích xuất từ cơ sở dữ liệu Olist. Mục tiêu là làm sạch dữ liệu, xử lý các dị biệt nghiệp vụ và chuẩn hóa cấu trúc để sẵn sàng nạp vào kho dữ liệu. Nội dung dưới đây trình bày chi tiết từng bước thực hiện kèm hình ảnh minh chứng từ kết quả chạy thực tế.

1.3.1 Khám phá và phát hiện vấn đề dữ liệu

Trước khi thực hiện biến đổi, tiến hành khảo sát cấu trúc thực thể, tính duy nhất của khóa chính (PK) và mối quan hệ giữa các bảng dữ liệu thô.

1.3.1.1 Kiểm tra số lượng dòng và khóa chính

Thực hiện kiểm tra số lượng bản ghi và kiểm định tính duy nhất của khóa chính trên từng bảng nhằm phát hiện sớm các trường hợp trùng lặp hoặc vi phạm ràng buộc toàn vẹn.

```
KHOA_CHINH = [('customers', ['customer_id']), ('sellers', ['seller_id']), ('products', ['product_id']), ('orders', ['order_id']),
    ('leads_qualified', ['mql_id']), ('leads_closed', ['mql_id']), ('order_reviews',  ['review_id']), ('category_translation', ['product_category_name']),
    ('order_items', ['order_id', 'order_item_id']), ('order_payments', ['order_id', 'payment_sequential']), ('geolocation', ['geolocation_zip_code_prefix']) ]

print(f"{'BANG':22s}  {'KHOA CHINH':36s}  {'SO DONG':>9s}  {'KHOA DUY NHAT':>13s}  KET LUAN")
print('-' * 118)

khong_hop_le = []
for ten, khoa in KHOA_CHINH:
    df = tables[ten]
    n_dong = len(df)
    n_khoa = df.drop_duplicates(subset=khoa).shape[0]
    ten_khoa = ' + '.join(khoa)
    if n_dong == n_khoa:
        ket_luan = 'khoa hop le'
    else:
        ket_luan = (f'{n_dong - n_khoa:,} dong trung')
        khong_hop_le.append(ten)
    print(f'{ten:22s}  {ten_khoa:36s}  {n_dong:>9,}  {n_khoa:>13,}  {ket_luan}')

print('-' * 118)
print(f'Bang co khoa hop le   : {len(KHOA_CHINH) - len(khong_hop_le)}/{len(KHOA_CHINH)}')
print(f'Bang can xu ly trung  : {", ".join(khong_hop_le) if khong_hop_le else "khong co"}')
```

[5] ✓ 0.1s

```
BANG                   KHOA CHINH                                SO DONG  KHOA DUY NHAT  KET LUAN
------------------------------------------------------------------------------------------------------------------------
customers              customer_id                                99,441         99,441  khoa hop le
sellers                seller_id                                   3,095          3,095  khoa hop le
products               product_id                                 32,951         32,951  khoa hop le
orders                 order_id                                   99,441         99,441  khoa hop le
leads_qualified        mql_id                                      8,000          8,000  khoa hop le
leads_closed           mql_id                                        842            842  khoa hop le
order_reviews          review_id                                  99,224         98,410  814 dong trung
category_translation   product_category_name                         71             71  khoa hop le
order_items            order_id + order_item_id                  112,650        112,650  khoa hop le
order_payments         order_id + payment_sequential             103,886        103,886  khoa hop le
geolocation            geolocation_zip_code_prefix             1,000,163         19,015  981,148 dong trung
------------------------------------------------------------------------------------------------------------------------
Bang co khoa hop le   : 9/11
Bang can xu ly trung  : order_reviews, geolocation
```

Hình 1.2: Kiểm tra số lượng dòng và tính hợp lệ của khóa chính trên các bảng thô

1.3.1.2 Kiểm tra dữ liệu nhiều dòng trên cùng một mã đơn hàng ở bảng thanh toán và đánh giá

Khảo sát quan hệ 1-N giữa bảng đơn hàng (orders) với bảng thanh toán (order_payments) và bảng đánh giá (order_reviews):

- Ở bảng order_payments, một đơn hàng có thể có nhiều dòng thanh toán do khách hàng chia nhiều đợt trả góp hoặc kết hợp nhiều hình thức thanh toán khác nhau (ví dụ: kết hợp thẻ tín dụng và voucher).
- Ở bảng order_reviews, một đơn hàng có thể nhận nhiều dòng đánh giá do hệ thống gửi khảo sát nhắc nhở hoặc khách hàng cập nhật lại phản hồi trải nghiệm dịch vụ.

```
pay_per_order = order_payments.groupby('order_id').size()
rev_per_order = order_reviews.groupby('order_id').size()
print('order_payments: max so dong / order =', pay_per_order.max(), '| so order co >1 dong payment =', (pay_per_order > 1).sum())
print('order_reviews : max so dong / order =', rev_per_order.max(), ' | so order co >1 dong review  =', (rev_per_order > 1).sum())
```

[6] ✓ 0.1s Python

```
order_payments: max so dong / order = 29 | so order co >1 dong payment = 2961
order_reviews : max so dong / order = 3  | so order co >1 dong review  = 547
```

Hình 1.3: Kiểm tra phân bố nhiều dòng dữ liệu trên cùng một mã đơn hàng

1.3.2 Xử lý giá trị thiếu

Các bước xử lý giá trị thiếu (NULL) được thực hiện trước khi xây dựng các bảng Dim và Fact, nhằm đảm bảo dữ liệu đưa vào các phép join không bị lỗi hoặc thất thoát bản ghi. Chiến lược xử lý được phân loại cụ thể theo từng nhóm tính chất trong Bảng 1.1.

Bảng 1.1: Chiến lược xử lý giá trị thiếu (NULL) theo nhóm đặc trưng

Nhóm dữ liệu	**Chiến lược xử lý**	**Cơ sở học thuật và nghiệp vụ**
Mang tín hiệu nghiệp vụ	**Giữ nguyên** NaN/NaT	Việc điền giá trị giả sẽ làm sai lệch trạng thái thực tế của quy trình đơn hàng.
Text tự do, optional	fillna(”)	Chuẩn hóa về chuỗi rỗng để tránh lỗi trong quá trình xử lý chuỗi.
Định danh / phân loại (FK, group-by)	fillna bằng placeholder	Gán nhãn quy ước (’outros’, ’unknown’) để không làm mất dòng khi join hoặc gom nhóm phân tích.
Cột số dùng tính toán định lượng	Impute bằng **median**	Dùng trung vị (median) thay vì mean vì phân phối kích thước/trọng lượng bị lệch bởi outlier; median phản ánh đúng xu thế trung tâm hơn.

1.3.2.1 Rà soát tổng quan giá trị thiếu trên tất cả bảng thô

Thực hiện rà soát tự động tỷ lệ giá trị thiếu trên toàn bộ các cột thuộc tính của 11 bảng thô.

```python
for name, df in tables.items():
    null_cols = df.isnull().sum()
    null_cols = null_cols[null_cols > 0]

    if not null_cols.empty:
        print(f"{name}:")
        for col, count in null_cols.items():
            pct = (count / len(df)) * 100
            print(f"  - {col:<32} : {count:>7,} ({pct:6.2f}%)")
        print()
```

[39] ✓ 0.4s

```
leads_closed:
  - business_segment                :       1 (  0.12%)
  - lead_type                       :       6 (  0.71%)
  - lead_behaviour_profile          :     177 ( 21.02%)
  - has_company                     :     779 ( 92.52%)
  - has_gtin                        :     778 ( 92.40%)
  - average_stock                   :     776 ( 92.16%)
  - business_type                   :      10 (  1.19%)
  - declared_product_catalog_size   :     773 ( 91.81%)

leads_qualified:
  - origin                          :      60 (  0.75%)

order_reviews:
  - review_comment_title            :  87,656 ( 88.34%)
  - review_comment_message          :  58,247 ( 58.70%)

orders:
  - order_approved_at               :     160 (  0.16%)
  - order_delivered_carrier_date    :   1,783 (  1.79%)
  - order_delivered_customer_date   :   2,965 (  2.98%)

products:
  - product_category_name           :     610 (  1.85%)
  - product_name_lenght             :     610 (  1.85%)
...
  - product_length_cm               :       2 (  0.01%)
  - product_height_cm               :       2 (  0.01%)
  - product_width_cm                :       2 (  0.01%)
```

Hình 1.4: Tổng quan phân bố giá trị thiếu trên tất cả bảng dữ liệu thô

1.3.2.2 Thay thế giá trị thiếu cột nguồn gốc trong bảng khách hàng tiềm năng

Cột origin trong bảng leads_qualified là biến phân loại kênh marketing (Organic Search, Paid Search, Social,...). Có 60 dòng bị thiếu thông tin do không ghi nhận được kênh tiếp cận. Giá trị thiếu được thay thế bằng placeholder 'unknown' để đảm bảo khi gom nhóm (group-by) đánh giá hiệu quả kênh tiếp thị không bị rơi vào nhóm khuyết danh.

```python
print('leads_qualified.origin NULL truoc xu ly:', leads_qualified['origin'].isna().sum())
leads_qualified['origin'] = leads_qualified['origin'].fillna('unknown')
print('leads_qualified.origin NULL sau xu ly  :', leads_qualified['origin'].isna().sum())
```

[8] ✓ 0.0s

```
leads_qualified.origin NULL truoc xu ly: 60
leads_qualified.origin NULL sau xu ly  : 0
```

Hình 1.5: Thay thế giá trị thiếu cột nguồn gốc tiếp thị bằng placeholder 'unknown'

1.3.2.3 Giữ nguyên giá trị thiếu các cột biểu mẫu marketing trong bảng chốt hợp đồng

Các cột như business_segment, lead_type, lead_behaviour_profile, has_company, average_stock, declared_product_catalog_size trong bảng leads_closed là thông tin do sales rep thu thập thủ công trong quá trình tư vấn và phần lớn là tùy chọn (optional), có tỷ lệ thiếu lên tới 92.5%.

> *Về mặt ngữ nghĩa, giá trị thiếu ở đây phản ánh trạng thái **"chưa thu thập được thông tin"**. Nếu thay thế bằng số 0 cho cột average_stock, dữ liệu sẽ bị hiểu sai lệch thành đối tác có tồn kho bằng 0. Do đó, việc giữ nguyên NaN là cần thiết để bảo toàn tính trung thực của dữ liệu khảo sát.*

```
leads_closed_optional_cols = [
    'business_segment', 'lead_type', 'lead_behaviour_profile', 'has_company',
    'has_gtin', 'average_stock', 'business_type', 'declared_product_catalog_size',
]
print(leads_closed[leads_closed_optional_cols].isna().sum())
print("-> giu nguyen NaN cho cac cot tren (NULL mang nghia 'chua thu thap')")
```

[9] ✓ 0.0s

```
business_segment                1
lead_type                       6
lead_behaviour_profile        177
has_company                   779
has_gtin                      778
average_stock                 776
business_type                  10
declared_product_catalog_size 773
dtype: int64
-> giu nguyen NaN cho cac cot tren (NULL mang nghia 'chua thu thap')
```

Hình 1.6: Thống kê và bảo toàn giá trị thiếu các cột biểu mẫu marketing trong bảng chốt hợp đồng

1.3.2.4 Thay thế giá trị thiếu tiêu đề và nội dung nhận xét trong bảng đánh giá

Cột tiêu đề (review_comment_title) có 87.656 dòng thiếu và nội dung (review_comment_message) có 58.247 dòng thiếu do khách hàng chỉ chấm điểm sao mà không để lại nhận xét văn bản. Các giá trị thiếu này được điền chuỗi rỗng " bằng phương thức fillna(") để các bước xử lý chuỗi và nạp dữ liệu không gặp lỗi định dạng.

```
print('NULL truoc xu ly:')
print(order_reviews[['review_comment_title', 'review_comment_message']].isna().sum())

order_reviews['review_comment_title'] = order_reviews['review_comment_title'].fillna('')
order_reviews['review_comment_message'] = order_reviews['review_comment_message'].fillna('')

print('NULL sau xu ly:')
print(order_reviews[['review_comment_title', 'review_comment_message']].isna().sum())
```

[10] ✓ 0.0s

```
NULL truoc xu ly:
review_comment_title      87656
review_comment_message    58247
dtype: int64
NULL sau xu ly:
review_comment_title      0
review_comment_message    0
dtype: int64
```

Hình 1.7: Thay thế giá trị thiếu tiêu đề và nội dung nhận xét bằng chuỗi rỗng

1.3.2.5 Giữ nguyên giá trị thiếu các cột ngày giao hàng và chuyển đổi kiểu dữ liệu

- Các cột order_approved_at, order_delivered_carrier_date, order_delivered_customer_date bị thiếu do đơn hàng **chưa đi hết chu trình giao hàng** (rơi vào trạng thái bị hủy canceled hoặc hết hàng unavailable).
- Giá trị thiếu ở đây là thông tin trạng thái nghiệp vụ thực tế, do đó không thay thế bằng ngày giả định.
- Thực hiện chuyển đổi sang kiểu datetime64[ns], các giá trị thiếu được giữ nguyên dưới dạng NaT để phục vụ tính toán chính xác thời gian vận chuyển (lead time) cho các đơn hàng đã hoàn tất.

```
date_cols = [
    'order_purchase_timestamp', 'order_approved_at', 'order_delivered_carrier_date',
    'order_delivered_customer_date', 'order_estimated_delivery_date',
]
for col in date_cols:
    orders[col] = pd.to_datetime(orders[col], errors='coerce')

print(orders[date_cols].isna().sum())
print('-> giu NaT, khong fillna (NULL o day la tin hieu trang thai don hang)')
```

[11] ✓ 0.1s

```
order_purchase_timestamp            0
order_approved_at                 160
order_delivered_carrier_date     1783
order_delivered_customer_date    2965
order_estimated_delivery_date       0
dtype: int64
-> giu NaT, khong fillna (NULL o day la tin hieu trang thai don hang)
```

Hình 1.8: Bảo toàn giá trị thiếu các mốc thời gian giao hàng sau khi chuyển đổi kiểu dữ liệu

1.3.2.6 Xử lý giá trị thiếu theo từng nhóm cột trong bảng sản phẩm

Bảng products có 610 sản phẩm thiếu thông tin phân loại và kích thước vật lý. Chiến lược xử lý được chia thành 3 nhóm cụ thể trong Bảng 1.2.

Bảng 1.2: Quy tắc xử lý giá trị thiếu theo từng nhóm cột trong bảng sản phẩm

Nhóm cột	Cách xử lý	Cơ sở kỹ thuật và nghiệp vụ
product_category_name	fillna('outros')	Gán vào nhóm danh mục chung "khác" (outros) theo chuẩn phân loại tiếng Bồ Đào Nha.
name_lenght, description_lenght, photos_qty	fillna(0)	Không có hình ảnh hoặc không có nội dung mô tả tương ứng với giá trị 0.
weight_g, length_cm, height_cm, width_cm	fillna(median)	Phân phối kích thước/trọng lượng bị lệch bởi outlier, dùng giá trị trung vị (median) để không làm méo mó các phân tích logistics.

```
print('NULL truoc xu ly:')
print(products.isna().sum()[products.isna().sum() > 0])

products['product_category_name'] = products['product_category_name'].fillna('outros')

for col in ['product_name_lenght', 'product_description_lenght', 'product_photos_qty']:
    products[col] = products[col].fillna(0)

for col in ['product_weight_g', 'product_length_cm', 'product_height_cm', 'product_width_cm']:
    products[col] = products[col].fillna(products[col].median())

null_after = products.isna().sum()
print('NULL sau xu ly:', null_after[null_after > 0].shape[0])
```

[12] ✓ 0.0s

```
NULL truoc xu ly:
product_category_name         610
product_name_lenght           610
product_description_lenght    610
product_photos_qty            610
product_weight_g                2
product_length_cm               2
product_height_cm               2
product_width_cm                2
dtype: int64
NULL sau xu ly: 0
```

Hình 1.9: Hiện thực hóa xử lý giá trị thiếu trên các nhóm cột trong bảng sản phẩm

1.3.2.7 Các bảng không có giá trị thiếu

Kiểm tra xác nhận các bảng customers, geolocation, order_items, order_payments, sellers và category_translation đạt độ hoàn thiện 100%, không chứa giá trị thiếu cần can thiệp.

1.3.3 Kiểm tra và xử lý dữ liệu trùng lặp

Kiểm tra trùng lặp (deduplication) được tiến hành trên từng bảng thô theo cả hai mức độ: trùng lặp khóa chính (PK conflict) và trùng lặp toàn dòng (full-row duplicate), thực hiện trước khi dựng các bảng Dim và Fact.

```
pk_map = {
    'customers': 'customer_id',
    'sellers': 'seller_id',
    'products': 'product_id',
    'orders': 'order_id',
    'geolocation': None,
    'leads_qualified': 'mql_id',
    'leads_closed': 'mql_id',
    'order_items': ['order_id', 'order_item_id'],
    'order_payments': ['order_id', 'payment_sequential'],
    'order_reviews': 'review_id',
    'category_translation': 'product_category_name',
}

for name, key in pk_map.items():
    df = tables[name]
    full_dup = df.duplicated().sum()
    if key is None:
        print(f'{name:22s} No PK | khong co PK don le de kiem tra rieng')
    else:
        pk_dup = df.duplicated(subset=key).sum()
        print(f'{name:22s} PK dup = {pk_dup:6d} ({pk_dup/len(df):.2%}) | full-row dup = {full_dup:7d} ({full_dup/len(df):.2%})')
```

[45] ✓ 1.4s

```
customers              PK dup =      0 (0.00%) | full-row dup =       0 (0.00%)
sellers                PK dup =      0 (0.00%) | full-row dup =       0 (0.00%)
products               PK dup =      0 (0.00%) | full-row dup =       0 (0.00%)
orders                 PK dup =      0 (0.00%) | full-row dup =       0 (0.00%)
geolocation            No PK | khong co PK don le de kiem tra rieng
leads_qualified        PK dup =      0 (0.00%) | full-row dup =       0 (0.00%)
leads_closed           PK dup =      0 (0.00%) | full-row dup =       0 (0.00%)
order_items            PK dup =      0 (0.00%) | full-row dup =       0 (0.00%)
order_payments         PK dup =      0 (0.00%) | full-row dup =       0 (0.00%)
order_reviews          PK dup =    814 (0.82%) | full-row dup =       0 (0.00%)
category_translation   PK dup =      0 (0.00%) | full-row dup =       0 (0.00%)
```

Hình 1.10: Rà soát bản ghi trùng lặp theo khóa chính và toàn dòng trên tất cả bảng thô

- Kết quả kiểm tra cho thấy bảng order_reviews có full-row duplicate = 0, nhưng xuất hiện xung đột khóa chính trên cột review_id ở một số dòng (do khách hàng chỉnh sửa phản hồi hoặc do độ trễ ghi nhận của hệ thống).
- Bảng Dim đánh giá yêu cầu mỗi mã review_id là duy nhất để đảm bảo quan hệ 1-1, do đó áp dụng deduplication giữ lại bản ghi phản hồi đầu tiên.

```
before = len(order_reviews)
order_reviews = order_reviews.drop_duplicates(subset='review_id', keep='first')
tables['order_reviews'] = order_reviews
after = len(order_reviews)

print('order_reviews truoc drop_duplicates:', before)
print('order_reviews sau drop_duplicates  :', after, f'(- {before - after} dong)')
print('review_id con trung sau xu ly      :', order_reviews['review_id'].duplicated().sum())
```

[46] ✓ 0.0s

```
order_reviews truoc drop_duplicates: 99224
order_reviews sau drop_duplicates  : 98410 (- 814 dong)
review_id con trung sau xu ly      : 0
```

Hình 1.11: Khử trùng lặp khóa chính review_id trong bảng đánh giá

1.3.4 Kiểm tra tính toàn vẹn tham chiếu trên dữ liệu thô

Kiểm tra khóa ngoại (FK) giữa các bảng thô nhằm phát hiện trước các bản ghi mồ côi (orphan records) cần xử lý khi thiết kế lược đồ.

1.3.4.1 Kiểm tra sự đầy đủ của danh mục sản phẩm trong bảng dịch danh mục

Đối chiếu giữa danh mục thực tế trong products và bảng từ điển dịch thuật category_ translation phát hiện có **3 danh mục sản phẩm** chưa có bản dịch tiếng Anh: 'outros' (do vừa điền placeholder), 'pc_gamer' và 'portateis_cozinha_e_preparadores_de_alimentos'. Vấn đề này được xử lý bổ sung khi xây dựng bảng Dim danh mục.

```
missing_categories = set(products['product_category_name'].dropna().unique()) - set(category_translation['product_category_name'])
print('So category trong products nhung KHONG co trong category_translation:', len(missing_categories))
print(sorted(missing_categories))
```

[47] ✓ 0.0s Python

```
So category trong products nhung KHONG co trong category_translation: 3
['outros', 'pc_gamer', 'portateis_cozinha_e_preparadores_de_alimentos']
```

Hình 1.12: Kiểm tra sự đầy đủ của danh mục sản phẩm trong bảng dịch danh mục

1.3.4.2 Kiểm tra sự tồn tại của mã người bán từ bảng chốt hợp đồng trong bảng người bán

Đối chiếu mã seller_id giữa bảng chốt hợp đồng (leads_closed) và bảng người bán (sellers) phát hiện có **462 / 842 seller_id (54.87%)** không tồn tại trong bảng sellers.

> ***Nhận xét bản chất nghiệp vụ:*** *Bảng sellers gốc trong hệ thống thương mại điện tử chỉ lưu trữ các seller đã có ít nhất 01 đơn hàng bán ra thực tế. Trong khi đó, 842 đối tác trong leads_closed là các lead vừa ký kết hợp đồng (closed deal). Trong số này, 380 seller (45.13%) đã kích hoạt gian hàng và phát sinh đơn hàng, còn 462 seller (54.87%) vẫn đang trong giai đoạn tiếp nhận hoặc chưa kịp bán được hàng trong thời gian thu thập dữ liệu. Đây là đặc trưng tự nhiên của phễu chuyển đổi tiếp thị (marketing conversion funnel), không phải lỗi dữ liệu. Do đó, các dòng này được giữ nguyên để phục vụ phân tích tỷ lệ kích hoạt đối tác.*

```
orphan_sellers = set(leads_closed['seller_id']) - set(sellers['seller_id'])
n_seller_id = leads_closed['seller_id'].nunique()
print(f'leads_closed co {len(orphan_sellers)}/{n_seller_id} seller_id KHONG ton tai trong sellers'
      f' ({len(orphan_sellers)/n_seller_id:.1%})')
```

[63] ✓ 0.0s

```
leads_closed co 462/842 seller_id KHONG ton tai trong sellers (54.9%)
```

Hình 1.13: Kiểm tra đối chiếu mã người bán giữa bảng chốt hợp đồng và bảng người bán

1.3.4.3 Kiểm tra tính khép kín của mã khách hàng tiềm năng

Kiểm tra khẳng định 100% các hợp đồng chốt được trong leads_closed đều có mã mql_id xuất phát hợp lệ từ bảng sàng lọc leads_qualified. Điều này đảm bảo khi thực hiện LEFT JOIN để dựng FACT_SELLER_LEAD sẽ không làm thất thoát bất kỳ bản ghi closed deal nào.

```
[17]
orphan_mql = set(leads_closed['mql_id']) - set(leads_qualified['mql_id'])
print(f'leads_closed co {len(orphan_mql)}/{len(leads_closed)} mql_id KHONG ton tai trong leads_qualified')
if len(orphan_mql) == 0:
    print('-> an toan, LEFT JOIN khong lam mat deal nao')
else:
    print('-> CANH BAO: se mat', len(orphan_mql), 'deal khi LEFT JOIN')
✓ 0.0s

leads_closed co 0/842 mql_id KHONG ton tai trong leads_qualified
-> an toan, LEFT JOIN khong lam mat deal nao
```

Hình 1.14: Xác nhận tính khép kín của định danh khách hàng tiềm năng trong phễu tiếp thị

1.3.5 Chuẩn hóa mã bưu chính (Zip Code Prefix)

Mã bưu chính tại Brazil gồm 8 chữ số, trong đó 5 chữ số đầu đại diện cho vùng địa lý đô thị. Do quá trình xuất dữ liệu thô sang kiểu số nguyên (integer), các mã bưu chính có chữ số 0 ở đầu (đặc biệt là khu vực São Paulo) bị mất số 0 dẫn đầu (ví dụ: 1001 thay vì 01001).

Tiến hành chuẩn hóa toàn bộ tiền tố zip code về chuỗi cố định 5 ký tự bằng hàm zfill(5) trên các bảng customers, sellers và geolocation, giải quyết triệt để lỗi phân mảnh khóa khi thực hiện lookup địa danh.

```
[18]
def normalize_zip(series):
    return series.astype(str).str.extract(r'(\d+)')[0].str.zfill(5)

customers['customer_zip_code_prefix'] = normalize_zip(customers['customer_zip_code_prefix'])
sellers['seller_zip_code_prefix'] = normalize_zip(sellers['seller_zip_code_prefix'])
geolocation['geolocation_zip_code_prefix'] = normalize_zip(geolocation['geolocation_zip_code_prefix'])
✓ 1.3s
```

Hình 1.15: Chuẩn hóa mã bưu chính về chuỗi ký tự cố định 5 chữ số

1.4 Thiết kế kho dữ liệu

1.4.1 Lựa chọn mô hình lược đồ

Kho dữ liệu được thiết kế theo mô hình **lược đồ bông tuyết (snowflake schema)**. Khác với lược đồ hình sao vốn giữ mỗi bảng chiều ở dạng một bảng phẳng duy nhất, lược đồ bông tuyết cho phép chuẩn hóa tiếp các phân cấp bên trong bảng chiều thành những bảng con riêng biệt.

Lựa chọn này xuất phát từ hai phân cấp có thật trong dữ liệu Olist:

- **Danh mục sản phẩm.** Mỗi sản phẩm thuộc một danh mục, và mỗi danh mục có thêm tên tiếng Anh tương ứng. Nếu gộp vào bảng chiều sản phẩm theo lược đồ hình sao, cặp tên danh mục song ngữ sẽ bị lặp lại trên toàn bộ hơn 32 nghìn dòng sản phẩm. Tách thành DIM_PRODUCT_CATEGORY giúp mỗi danh mục chỉ được lưu một lần.
- **Địa điểm.** Khách hàng và người bán cùng tham chiếu đến một tập mã bưu chính và tọa độ địa lý. Tách thành DIM_LOCATION dùng chung cho cả hai bảng chiều giúp tránh nhân đôi hơn một triệu dòng tọa độ, đồng thời bảo đảm hai bảng chiều luôn hiểu địa danh theo cùng một chuẩn.

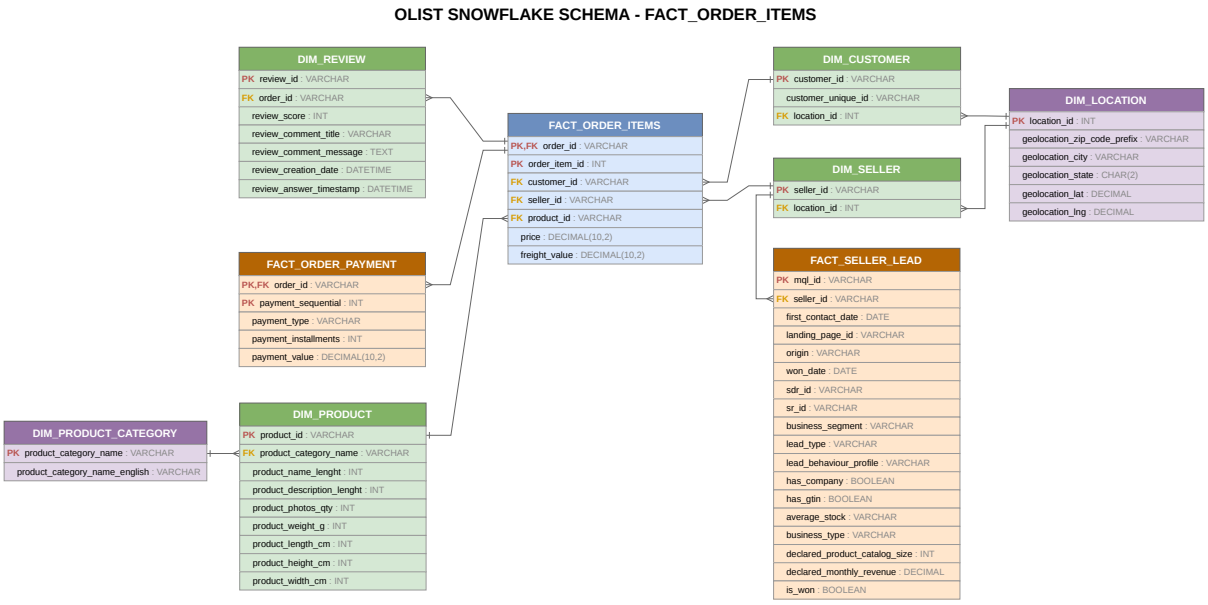
Cần lưu ý rằng lược đồ còn có ba bảng sự kiện khác nhau (chi tiết đơn hàng, lần thanh toán và khách hàng tiềm năng) cùng chia sẻ các bảng chiều, nên xét chặt chẽ đây là một *lược đồ chòm sao* với các bảng chiều đã được bông tuyết hóa. Phần trình bày dưới đây vẫn gọi chung là lược đồ bông tuyết theo đặc trưng nổi bật nhất của thiết kế.

1.4.2 Lược đồ kho dữ liệu

Lược đồ đích gồm **ba bảng sự kiện** và **sáu bảng chiều**, được thể hiện trong Hình 1.16.

FACT_ORDER_ITEMS giữ vai trò bảng sự kiện trung tâm, liên kết tới ba bảng chiều qua customer_id, seller_id và product_id. Hai nhánh bông tuyết nằm ở phía sau các bảng chiều này: DIM_PRODUCT trỏ tiếp tới DIM_PRODUCT_CATEGORY qua product_category_name, còn DIM_CUSTOMER và DIM_SELLER cùng trỏ tới DIM_LOCATION qua khóa thay thế location_id.

Ba quan hệ còn lại không đi qua bảng chiều: FACT_ORDER_PAYMENT và DIM_REVIEW cùng tham chiếu order_id của FACT_ORDER_ITEMS, còn FACT_SELLER_LEAD tham chiếu seller_id của DIM_SELLER. Toàn bộ lược đồ dùng khóa tự nhiên của dữ liệu gốc, ngoại trừ location_id là khóa thay thế được sinh thêm vì tiền tố mã bưu chính không có tính duy nhất trong bảng tọa độ.



Hình 1.16: Lược đồ bông tuyết của kho dữ liệu Olist sau tiền xử lý

1.4.3 Xây dựng và mô tả chi tiết các bảng Chiều (Dimension Tables)

Để bảo đảm tính toàn vẹn tham chiếu khóa ngoại, các bảng chiều (Dimension) được tạo trước từ dữ liệu nguồn đã qua làm sạch để làm cơ sở tra cứu cho các bảng sự kiện. Dưới đây là quy trình xây dựng, cài đặt và từ điển dữ liệu chi tiết cho từng bảng chiều theo thứ tự phụ thuộc.

1.4.3.1 Bảng chiều địa điểm (DIM_LOCATION)

Bảng DIM_LOCATION giữ nguyên toàn bộ 1.000.163 dòng tọa độ gốc để bảo toàn thông tin không gian, bổ sung thêm khóa thay thế tự tăng location_id (Surrogate Key) từ 1 đến N. Do tiền tố mã bưu chính (geolocation_zip_code_prefix) không có tính duy nhất trong bảng tọa độ (một mã zip có thể ứng với nhiều điểm khảo sát tọa độ khác nhau), các bảng chiều khách hàng và người bán sẽ thực hiện liên kết thông qua khóa thay thế location_id này.

```
dim_location = geolocation.copy()
dim_location.insert(0, 'location_id', range(1, len(dim_location) + 1))

print('DIM_LOCATION:', dim_location.shape)
```

[19] ✓ 0.0s

```
DIM_LOCATION: (1000163, 6)
```

Hình 1.17: Hiện thực hóa xây dựng bảng chiều địa điểm DIM_LOCATION

Bảng 1.3: Các thuộc tính của bảng DIM_LOCATION

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
location_id	integer	Khóa chính thay thế của bảng chiều địa điểm, đánh số tuần tự
geolocation_zip_code_prefix	string	Năm chữ số đầu của mã bưu chính, đã chuẩn hóa thành chuỗi 5 ký tự
geolocation_lat	float	Vĩ độ của điểm tọa độ
geolocation_lng	float	Kinh độ của điểm tọa độ
geolocation_city	string	Tên thành phố ứng với tọa độ
geolocation_state	string	Mã bang ứng với tọa độ

1.4.3.2 Bảng chiều danh mục sản phẩm (DIM_PRODUCT_CATEGORY)

Bảng DIM_PRODUCT_CATEGORY được xây dựng từ bảng category_translation, bổ sung thêm 3 danh mục còn thiếu đã phát hiện ở bước 1.3 (bao gồm cả 'outros'). Đối với các danh mục bổ sung, sử dụng chính tên gốc tiếng Bồ Đào Nha làm placeholder cho cột tiếng Anh, tạo thành từ điển phân loại hoàn chỉnh gồm 74 dòng.

```
dim_product_category = category_translation.copy()

if missing_categories:
    extra = pd.DataFrame({
        'product_category_name': sorted(missing_categories),
        'product_category_name_english': np.nan,
    })
    dim_product_category = pd.concat([dim_product_category, extra], ignore_index=True)

dim_product_category['product_category_name_english'] = (
    dim_product_category['product_category_name_english']
    .fillna(dim_product_category['product_category_name'])
)

print('DIM_PRODUCT_CATEGORY:', dim_product_category.shape, '| NULL con lai:', dim_product_category.isna().sum().sum())
```

[20] ✓ 0.0s

```
DIM_PRODUCT_CATEGORY: (74, 2) | NULL con lai: 0
```

Hình 1.18: Xây dựng bảng chiều danh mục sản phẩm song ngữ

Bảng 1.4: Các thuộc tính của bảng DIM_PRODUCT_CATEGORY

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
product_category_name	string	Tên danh mục sản phẩm bằng tiếng Bồ Đào Nha, khóa chính
product_category_name_english	string	Tên danh mục tương ứng bằng tiếng Anh

1.4.3.3 Bảng chiều sản phẩm (DIM_PRODUCT)

Bảng DIM_PRODUCT gồm 32.951 sản phẩm duy nhất với 9 thuộc tính mô tả đặc tính vật lý và danh mục sản phẩm đã được làm sạch ở bước xử lý giá trị khuyết thiếu. Bảng liên kết khóa

ngoại product_category_name tới bảng DIM_PRODUCT_CATEGORY.

```
dim_product = products[[
    'product_id', 'product_category_name', 'product_name_lenght', 'product_description_lenght',
    'product_photos_qty', 'product_weight_g', 'product_length_cm', 'product_height_cm', 'product_width_cm',
]].copy()

print('DIM_PRODUCT:', dim_product.shape)
```

[21] ✓ 0.0s

```
DIM_PRODUCT: (32951, 9)
```

Hình 1.19: Cấu trúc bảng chiều sản phẩm sau khi làm sạch

Bảng 1.5: Các thuộc tính của bảng DIM_PRODUCT

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
product_id	string	Mã sản phẩm, khóa chính
product_category_name	string	Tên danh mục sản phẩm bằng tiếng Bồ Đào Nha, khóa chính (khóa ngoại tới DIM_PRODUCT_CATEGORY)
product_name_lenght	float	Số ký tự trong tên sản phẩm
product_description_lenght	float	Số ký tự trong phần mô tả sản phẩm
product_photos_qty	float	Số lượng ảnh đăng kèm sản phẩm
product_weight_g	float	Khối lượng sản phẩm, đơn vị gam
product_length_cm	float	Chiều dài kiện hàng, đơn vị cm
product_height_cm	float	Chiều cao kiện hàng, đơn vị cm
product_width_cm	float	Chiều rộng kiện hàng, đơn vị cm

1.4.3.4 Bảng chiều khách hàng (DIM_CUSTOMER)

Bảng DIM_CUSTOMER định danh khách hàng gắn với từng đơn hàng (customer_id) và người mua duy nhất (customer_unique_id). Để ánh xạ chính xác khóa ngoại location_id từ bảng địa điểm cho khách hàng mà không để lại giá trị khuyết, quy trình lookup phân cấp 3 bước được áp dụng:

1. **Bước 1 (Khớp chính xác bộ ba):** Chuẩn hóa tên thành phố, bang và thực hiện join theo bộ ba giá trị (zip, city, state) sau khi chuyển về chữ thường.
2. **Bước 2 (Fallback lookup theo Zip):** Với các dòng không khớp do sai lệch ký tự tên thành phố, áp dụng phương án dự phòng lookup chỉ theo mã zip.
3. **Bước 3 (Bổ sung địa danh mới):** Với số lượng nhỏ khách hàng có mã bưu chính hoàn toàn chưa xuất hiện trong bảng khảo sát tọa độ, hệ thống tự động chèn dòng địa danh mới vào DIM_LOCATION (với tọa độ để trống) và gán ngược lại location_id.

Quy trình này bảo đảm 100% khách hàng đều có khóa ngoại location_id hợp lệ, đạt tỷ lệ orphan 0%.

```python
geo_lookup = (
    dim_location
    .assign(
        _city_lc=dim_location['geolocation_city'].str.lower(),
        _state_uc=dim_location['geolocation_state'].str.upper(),
    )
    .drop_duplicates(subset=['geolocation_zip_code_prefix', '_city_lc', '_state_uc'], keep='first')
    [['location_id', 'geolocation_zip_code_prefix', '_city_lc', '_state_uc']]
)

# Bang tra cuu fallback: khop theo zip only (dung khi khong tim duoc theo ca 3 cot)
zip_to_location = (
    dim_location
    .drop_duplicates(subset=['geolocation_zip_code_prefix'], keep='first')
    .set_index('geolocation_zip_code_prefix')['location_id']
)

customers['_city_lc'] = customers['customer_city'].str.lower()
customers['_state_uc'] = customers['customer_state'].str.upper()

dim_customer = customers.merge(
    geo_lookup,
    left_on=['customer_zip_code_prefix', '_city_lc', '_state_uc'],
    right_on=['geolocation_zip_code_prefix', '_city_lc', '_state_uc'],
    how='left',
)[['customer_id', 'customer_unique_id', 'location_id', 'customer_zip_code_prefix', 'customer_city', 'customer_state']]
```

Hình 1.20: Quy trình ánh xạ khóa địa điểm cho bảng chiều khách hàng (Bước 1)

```python
missing_before = dim_customer['location_id'].isna().sum()

# Xu ly NULL: fallback khop theo zip-only cho cac dong chua tim duoc theo (zip, city, state)
dim_customer['location_id'] = dim_customer['location_id'].fillna(
    dim_customer['customer_zip_code_prefix'].map(zip_to_location)
)

missing_after_fallback = dim_customer['location_id'].isna().sum()

# Xu ly NULL con lai: zip cua customer khong ton tai trong geolocation.csv
still_missing = dim_customer['location_id'].isna()
if still_missing.any():
    new_locations = (
        dim_customer.loc[still_missing, ['customer_zip_code_prefix', 'customer_city', 'customer_state']]
        .drop_duplicates(subset='customer_zip_code_prefix')
        .rename(columns={
            'customer_zip_code_prefix': 'geolocation_zip_code_prefix',
            'customer_city': 'geolocation_city',
            'customer_state': 'geolocation_state',
        })
    )
    new_locations['geolocation_lat'] = np.nan
    new_locations['geolocation_lng'] = np.nan
    next_id = dim_location['location_id'].max() + 1
    new_locations.insert(0, 'location_id', range(next_id, next_id + len(new_locations)))
    new_locations = new_locations[dim_location.columns]
```

Hình 1.21: Thực hiện fallback lookup và bổ sung địa danh cho khách hàng (Bước 2)

```
    dim_location = pd.concat([dim_location, new_locations], ignore_index=True)
    zip_to_location = pd.concat([zip_to_location, new_locations.set_index('geolocation_zip_code_prefix')['location_id']])

    dim_customer['location_id'] = dim_customer['location_id'].fillna(
        dim_customer['customer_zip_code_prefix'].map(zip_to_location)
    )
    print(f'Da them {len(new_locations)} dong moi vao DIM_LOCATION tu zip cua customer (lat/lng = NaN, chua co du lieu GPS thuc)')

dim_customer = dim_customer[['customer_id', 'customer_unique_id', 'location_id']]

missing_final = dim_customer['location_id'].isna().sum()
print('DIM_CUSTOMER:', dim_customer.shape)
print('khong tim duoc location_id (zip+city+state)         :', missing_before)
print('khong tim duoc location_id sau fallback zip          :', missing_after_fallback)
print('khong tim duoc location_id sau khi bo sung DIM_LOCATION:', missing_final, '<- ky vong = 0')
```

[22] ✓ 0.7s Python

```
Da them 157 dong moi vao DIM_LOCATION tu zip cua customer (lat/lng = NaN, chua co du lieu GPS thuc)
DIM_CUSTOMER: (99441, 3)
khong tim duoc location_id (zip+city+state)         : 318
khong tim duoc location_id sau fallback zip          : 278
khong tim duoc location_id sau khi bo sung DIM_LOCATION: 0 <- ky vong = 0
```

Hình 1.22: Xác nhận toàn vẹn khóa ngoại location_id trên bảng khách hàng (Bước 3)

Bảng 1.6: Các thuộc tính của bảng DIM_CUSTOMER

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
customer_id	string	Mã khách hàng gắn với từng đơn hàng, khóa chính
customer_unique_id	string	Mã nhận diện duy nhất của một người mua, dùng để phát hiện khách mua lặp lại
location_id	float	Khóa chính thay thế của bảng chiều địa điểm, đánh số tuần tự (khóa ngoại tới DIM_LOCATION)

1.4.3.5 Bảng chiều người bán (DIM_SELLER)

Bảng DIM_SELLER định danh người bán (seller_id). Tương tự bảng khách hàng, quy trình lookup phân cấp 3 bước được áp dụng bằng cách tái sử dụng bảng tra cứu zip_to_location đã hoàn thiện. Toàn bộ 3.095 người bán đều được gán location_id thành công mà không để lại giá trị NULL.

```
sellers['_city_lc'] = sellers['seller_city'].str.lower()
sellers['_state_uc'] = sellers['seller_state'].str.upper()

dim_seller = sellers.merge(
    geo_lookup,
    left_on=['seller_zip_code_prefix', '_city_lc', '_state_uc'],
    right_on=['geolocation_zip_code_prefix', '_city_lc', '_state_uc'],
    how='left',
)[['seller_id', 'location_id', 'seller_zip_code_prefix', 'seller_city', 'seller_state']]

missing_before = dim_seller['location_id'].isna().sum()

# Xu ly NULL: fallback khop theo zip-only, dung lai bang tra cuu zip_to_location tu muc 12
dim_seller['location_id'] = dim_seller['location_id'].fillna(
    dim_seller['seller_zip_code_prefix'].map(zip_to_location)
)

missing_after_fallback = dim_seller['location_id'].isna().sum()
```

Hình 1.23: Ánh xạ khóa địa điểm cho bảng chiều người bán (Phần 1)

```python
# Xu ly NULL con lai: zip cua seller khong ton tai trong geolocation.csv
still_missing = dim_seller['location_id'].isna()
if still_missing.any():
    new_locations = (
        dim_seller.loc[still_missing, ['seller_zip_code_prefix', 'seller_city', 'seller_state']]
        .drop_duplicates(subset='seller_zip_code_prefix')
        .rename(columns={
            'seller_zip_code_prefix': 'geolocation_zip_code_prefix',
            'seller_city': 'geolocation_city',
            'seller_state': 'geolocation_state',
        })
    )
    new_locations['geolocation_lat'] = np.nan
    new_locations['geolocation_lng'] = np.nan
    next_id = dim_location['location_id'].max() + 1
    new_locations.insert(0, 'location_id', range(next_id, next_id + len(new_locations)))
    new_locations = new_locations[dim_location.columns]

    dim_location = pd.concat([dim_location, new_locations], ignore_index=True)
    zip_to_location = pd.concat([zip_to_location, new_locations.set_index('geolocation_zip_code_prefix')['location_id']])

    dim_seller['location_id'] = dim_seller['location_id'].fillna(
        dim_seller['seller_zip_code_prefix'].map(zip_to_location)
    )
    print(f'Da them {len(new_locations)} dong moi vao DIM_LOCATION tu zip cua seller (lat/lng = NaN, chua co du lieu GPS thuc)')
```

Hình 1.24: Ánh xạ khóa địa điểm cho bảng chiều người bán (Phần 2)

```python
missing_final = dim_seller['location_id'].isna().sum()
print('DIM_SELLER:', dim_seller.shape)
print('khong tim duoc location_id (zip+city+state)          :', missing_before)
print('khong tim duoc location_id sau fallback zip           :', missing_after_fallback)
print('khong tim duoc location_id sau khi bo sung DIM_LOCATION:', missing_final)
print('DIM_LOCATION cuoi cung (sau bo sung tu ca customer & seller):', dim_location.shape)
```

[23] ✓ 0.0s Python

```
Da them 5 dong moi vao DIM_LOCATION tu zip cua seller (lat/lng = NaN, chua co du lieu GPS thuc)
DIM_SELLER: (3095, 2)
khong tim duoc location_id (zip+city+state)          : 137
khong tim duoc location_id sau fallback zip           : 5
khong tim duoc location_id sau khi bo sung DIM_LOCATION: 0
DIM_LOCATION cuoi cung (sau bo sung tu ca customer & seller): (1000325, 6)
```

Hình 1.25: Khẳng định tính hoàn thiện không khuyết thiếu của khóa ngoại ở bảng chiều người bán

Bảng 1.7: Các thuộc tính của bảng DIM_SELLER

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
seller_id	string	Mã người bán, khóa chính
location_id	float	Khóa chính thay thế của bảng chiều địa điểm, đánh số tuần tự (khóa ngoại tới DIM_LOCATION)

1.4.3.6 Bảng chiều đánh giá (DIM_REVIEW)

Bảng DIM_REVIEW lưu trữ 98.410 phiếu đánh giá trải nghiệm dịch vụ của khách hàng. Quá trình xử lý bao gồm:

- Khử trùng lặp khóa chính review_id (giữ lại bản ghi phản hồi đầu tiên).
- Chuyển đổi các cột ngày tạo phiếu (review_creation_date) và thời điểm trả lời (review_answer_timestamp) sang kiểu thời gian (datetime).

- Tiêu đề và nội dung nhận xét khuyết thiếu đã được thay thế bằng chuỗi rỗng ở bước làm sạch dữ liệu, liên kết tới đơn hàng qua khóa ngoại order_id.

```
dim_review = order_reviews.drop_duplicates(subset='review_id').copy()
dim_review['review_creation_date'] = pd.to_datetime(dim_review['review_creation_date'], errors='coerce')
dim_review['review_answer_timestamp'] = pd.to_datetime(dim_review['review_answer_timestamp'], errors='coerce')

dim_review = dim_review[[
    'review_id', 'order_id', 'review_score', 'review_comment_title',
    'review_comment_message', 'review_creation_date', 'review_answer_timestamp',
]]

print('DIM_REVIEW:', dim_review.shape)
```

[24] ✓ 0.0s

```
DIM_REVIEW: (98410, 7)
```

Hình 1.26: Cấu trúc hoàn thiện của bảng chiều đánh giá DIM_REVIEW

Bảng 1.8: Các thuộc tính của bảng DIM_REVIEW

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
review_id	string	Mã phiếu đánh giá, khóa chính
order_id	string	Mã đơn hàng (khóa ngoại tới FACT_ORDER_ITEMS)
review_score	integer	Điểm đánh giá của khách, từ 1 đến 5
review_comment_title	string	Tiêu đề nhận xét do khách nhập
review_comment_message	string	Nội dung nhận xét do khách nhập
review_creation_date	datetime	Ngày phiếu khảo sát được gửi tới khách hàng
review_answer_timestamp	datetime	Thời điểm khách hàng trả lời khảo sát

1.4.4 Xây dựng và mô tả chi tiết các bảng Sự kiện (Fact Tables)

Sau khi toàn bộ các bảng chiều đã hoàn thiện và sẵn sàng làm từ điển tra cứu, ba bảng sự kiện (Fact) được khởi tạo theo các mức độ chi tiết (grain) và nghiệp vụ đo lường riêng biệt.

1.4.4.1 Bảng sự kiện chi tiết đơn hàng (FACT_ORDER_ITEMS)

Bảng sự kiện trung tâm FACT_ORDER_ITEMS có grain ở mức chi tiết nhất (atomic grain): mỗi dòng biểu diễn một sản phẩm trong đơn hàng, xác định bởi khóa chính kết hợp (order_id, order_item_id). Bảng được tạo bằng cách nối bảng chi tiết mặt hàng (order_items) với bảng đơn hàng (orders) để bổ sung khóa ngoại customer_id, đồng thời liên kết tới DIM_SELLER và DIM_PRODUCT. Bảng lưu trữ hai độ đo tài chính cốt lõi: đơn giá (price) và cước phí vận chuyển (freight_value), với tổng quy mô 112.650 bản ghi.

```
fact_order_items = order_items.merge(
    orders[['order_id', 'customer_id']], on='order_id', how='left',
)[['order_id', 'order_item_id', 'customer_id', 'seller_id', 'product_id', 'price', 'freight_value']]

print('FACT_ORDER_ITEMS:', fact_order_items.shape)
```

[25] ✓ 0.0s

```
FACT_ORDER_ITEMS: (112650, 7)
```

Hình 1.27: Xây dựng bảng sự kiện chi tiết đơn hàng FACT_ORDER_ITEMS

Bảng 1.9: Các thuộc tính của bảng FACT_ORDER_ITEMS

Tên thuộc tính	**Kiểu dữ liệu**	**Ý nghĩa**
order_id	string	Mã đơn hàng, khóa chính cùng với order_item_id
order_item_id	integer	Số thứ tự của sản phẩm trong cùng một đơn hàng
customer_id	string	Mã khách hàng gắn với từng đơn hàng, khóa chính (khóa ngoại tới DIM_CUSTOMER)
seller_id	string	Mã người bán, khóa chính (khóa ngoại tới DIM_SELLER)
product_id	string	Mã sản phẩm, khóa chính (khóa ngoại tới DIM_PRODUCT)
price	float	Giá bán của sản phẩm, đơn vị Real Brazil (BRL)
freight_value	float	Phí vận chuyển phân bổ cho dòng sản phẩm này (BRL)

1.4.4.2 Bảng sự kiện thanh toán (FACT_ORDER_PAYMENT)

Bảng sự kiện thanh toán FACT_ORDER_PAYMENT có grain ở mức từng giao dịch thanh toán, xác định bởi cặp khóa (order_id, payment_sequential). Bảng được tách riêng khỏi bảng chi tiết mặt hàng do khác biệt về mức độ chi tiết (một đơn hàng có thể có nhiều đợt hoặc nhiều phương thức thanh toán khác nhau). Thiết kế này giúp theo dõi chính xác dòng tiền thanh toán (payment_value) và số kỳ trả góp (payment_installments) mà không gây dư thừa hay nhân bản doanh thu đơn hàng.

```
fact_order_payment = order_payments[[
    'order_id', 'payment_sequential', 'payment_type', 'payment_installments', 'payment_value',
]].copy()

print('FACT_ORDER_PAYMENT:', fact_order_payment.shape)
```

[26] ✓ 0.0s

```
FACT_ORDER_PAYMENT: (103886, 5)
```

Hình 1.28: Xây dựng bảng sự kiện thanh toán FACT_ORDER_PAYMENT

Bảng 1.10: Các thuộc tính của bảng FACT_ORDER_PAYMENT

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
order_id	string	Mã đơn hàng (khóa ngoại tới FACT_ORDER_ITEMS)
payment_sequential	integer	Số thứ tự của lần thanh toán trong cùng một đơn hàng
payment_type	string	Hình thức thanh toán
payment_installments	integer	Số kỳ trả góp của lần thanh toán
payment_value	float	Số tiền của lần thanh toán (BRL)

1.4.4.3 Bảng sự kiện khách hàng tiềm năng (FACT_SELLER_LEAD)

Bảng sự kiện phễu tiếp thị FACT_SELLER_LEAD theo dõi quá trình chuyển đổi đối tác kinh doanh: mỗi dòng tương ứng với một khách hàng tiềm năng (lead), xác định bởi khóa chính mql_id. Bảng được xây dựng bằng phép LEFT JOIN từ 8.000 leads đủ điều kiện (leads_qualified) sang bảng chốt hợp đồng (leads_closed) theo mql_id. Cờ nhị phân is_won được tạo để phân biệt lead đã chốt hợp đồng thành công thành người bán (seller_id). Các trường dữ liệu tự khai (doanh thu kỳ vọng, phân khúc, quy mô kho bãi...) được giữ nguyên để phục vụ phân tích hiệu quả phễu chuyển đổi.

```
fact_seller_lead = leads_qualified.merge(leads_closed, on='mql_id', how='left')
fact_seller_lead['is_won'] = fact_seller_lead['seller_id'].notna()

print('FACT_SELLER_LEAD:', fact_seller_lead.shape)
```

[27] ✓ 0.0s

```
FACT_SELLER_LEAD: (8000, 18)
```

Hình 1.29: Xây dựng bảng sự kiện phễu tiếp thị FACT_SELLER_LEAD

Bảng 1.11: Các thuộc tính của bảng FACT_SELLER_LEAD

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
mql_id	string	Mã khách hàng tiềm năng đã qua sàng lọc, khóa chính
first_contact_date	string	Ngày lead điền biểu mẫu liên hệ lần đầu
landing_page_id	string	Mã trang đích mà lead truy cập
origin	string	Kênh đưa lead đến sàn
seller_id	string	Mã người bán, khóa chính (khóa ngoại tới DIM_SELLER)
sdr_id	string	Mã nhân viên phát triển kinh doanh tiếp nhận lead
sr_id	string	Mã nhân viên bán hàng chốt hợp đồng
won_date	string	Thời điểm lead chính thức trở thành người bán
business_segment	string	Phân khúc ngành hàng của người bán
lead_type	string	Loại hình lead
lead_behaviour_profile	string	Hồ sơ hành vi của lead
has_company	float	Lead đã có pháp nhân đăng ký hay chưa
has_gtin	float	Sản phẩm của lead đã có mã vạch GTIN hay chưa

(xem tiếp trang sau)

Bảng 1.11 (tiếp theo)

Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
average_stock	string	Khoảng tồn kho trung bình do lead tự khai
business_type	string	Loại hình kinh doanh
declared_product_catalog_size	float	Số lượng sản phẩm trong danh mục do lead tự khai
declared_monthly_revenue	float	Doanh thu hằng tháng do lead tự khai (BRL)
is_won	boolean	Cờ đánh dấu lead đã chuyển đổi thành người bán

1.4.5 Kiểm tra tính toàn vẹn tham chiếu

Sau khi hoàn thiện các bảng Fact và Dim, tính toàn vẹn tham chiếu khóa ngoại (Foreign Key Integrity) giữa các bảng sự kiện và bảng chiều trên lược đồ đích được kiểm tra tự động bằng mã kiểm thử (Hình 1.30).

```
def check_fk(fact_df, fact_col, dim_df, dim_col, label):
    orphan = set(fact_df[fact_col].dropna()) - set(dim_df[dim_col].dropna())
    if len(orphan) > 0:
        total = fact_df[fact_col].nunique()
        pct = (len(orphan) / total) * 100
        print(f'{label} = {len(orphan):,d} / {total:,d} ({pct:.2f}%)')
    return orphan

check_fk(fact_order_items, 'customer_id', dim_customer, 'customer_id', 'FACT_ORDER_ITEMS.customer_id -> DIM_CUSTOMER')
check_fk(fact_order_items, 'seller_id',   dim_seller,   'seller_id',   'FACT_ORDER_ITEMS.seller_id -> DIM_SELLER')
check_fk(fact_order_items, 'product_id',  dim_product,  'product_id',  'FACT_ORDER_ITEMS.product_id -> DIM_PRODUCT')
check_fk(dim_product,   'product_category_name', dim_product_category, 'product_category_name', 'DIM_PRODUCT.product_category_name -> DIM_PRODUCT_CATEGORY')
check_fk(dim_customer, 'location_id', dim_location, 'location_id', 'DIM_CUSTOMER.location_id -> DIM_LOCATION')
check_fk(dim_seller,   'location_id', dim_location, 'location_id', 'DIM_SELLER.location_id -> DIM_LOCATION')
check_fk(fact_order_payment, 'order_id', fact_order_items, 'order_id', 'FACT_ORDER_PAYMENT.order_id -> FACT_ORDER_ITEMS.order_id')
check_fk(dim_review,   'order_id', fact_order_items, 'order_id', 'DIM_REVIEW.order_id -> FACT_ORDER_ITEMS.order_id')
check_fk(fact_seller_lead, 'seller_id', dim_seller, 'seller_id', 'FACT_SELLER_LEAD.seller_id -> DIM_SELLER')
```

```
[32] ✓ 0.5s                                                                    Python
...  FACT_ORDER_PAYMENT.order_id -> FACT_ORDER_ITEMS.order_id = 775 / 99,440 (0.78%)
     DIM_REVIEW.order_id -> FACT_ORDER_ITEMS.order_id = 725 / 98,167 (0.74%)
     FACT_SELLER_LEAD.seller_id -> DIM_SELLER = 462 / 842 (54.87%)
```

Hình 1.30: Kiểm tra tính toàn vẹn tham chiếu giữa các bảng sự kiện và bảng chiều

Kết quả kiểm tra xác nhận phần lớn các khóa ngoại liên kết tới các bảng chiều khách hàng, người bán, sản phẩm và địa điểm đều đạt tính toàn vẹn tuyệt đối 100% (0 bản ghi orphan). Ba mối quan hệ ghi nhận bản ghi orphan được giải thích thỏa đáng theo đặc thù nghiệp vụ như sau:

1. **FACT_ORDER_PAYMENT.order_id → FACT_ORDER_ITEMS.order_id (775 orphan / 99.440 đơn, 0.78%):** Khách hàng đã thanh toán thành công qua cổng thanh toán, nhưng đơn hàng sau đó bị hủy (canceled) hoặc không có hàng giao (unavailable) trước khi chi tiết sản phẩm được ghi nhận trong kho. Đây là hiện tượng nghiệp vụ thực tế của dòng tiền, không phải lỗi ETL. Dữ liệu này được giữ nguyên nhằm phục vụ bài toán đối soát tài chính và hoàn tiền.
2. **DIM_REVIEW.order_id → FACT_ORDER_ITEMS.order_id (725 orphan / 98.167 đơn, 0.74%):** Tương tự trường hợp thanh toán, khách hàng gửi đánh giá trải nghiệm cho các đơn hàng đã bị hủy hoặc sự cố trước khi xử lý mặt hàng.

3. **FACT_SELLER_LEAD.seller_id → DIM_SELLER.seller_id (462 orphan / 842 đối tác, 54.87%):** Như đã phân tích chi tiết tại mục 1.3, 462 đối tác đã ký hợp đồng thành công nhưng chưa từng phát sinh đơn hàng nào trên sàn, do đó chưa xuất hiện trong danh mục người bán vận hành.

2 Quá trình xây dựng kho dữ liệu (SSIS)

2.1 Chuẩn bị các công cụ

2.2 Chuẩn bị cơ sở dữ liệu

2.3 Tạo mới project SSIS

2.4 Tạo bảng DIM và FACT

2.4.1 Load dữ liệu gốc từ files .csv vào Data Flow

2.4.2 Bảng DIM_Time

2.4.3 Bảng DIM_Language

2.4.4 Bảng DIM_Artist

2.4.5 Bảng DIM_Album

2.4.6 Bảng DIM_AudioFeature

2.4.7 Bảng FACT_Track

2.4.8 Tạo khóa ngoại từ bảng FACT đến các bảng DIM

2.4.9 Chạy dự án SSIS

2.5 Kiểm tra dữ liệu các bảng

2.6 Lược đồ sau khi hoàn thành